Software Design

BIS605

Software Development Technologies for Digital Business

Asst. Prof. Dr. Worarat Krathu

Resources

- https://www.tutorialspoint.com/software_engineering/software_design_basics.htm
- https://www.javatpoint.com/
- Naveen Sagayaselvaraj, Software Design, published on Sep 24, 2016 at https://www.slideshare.net/
- MeghajKumarMallick, Design Model & User Interface Design in Software Engineering, published on Apr 16, 2020 at https://www.slideshare.net/
- Naveen Sagayaselvaraj, User interface design, published on Oct 8, 2016 at https://www.slideshare.net/

Software Design

- A process to transform user requirements into some forms, models, specifications which help the programmer in software coding and implementation

- Software design levels
  - High-level design – architectural design which identifies related components and their interactions. Sometimes, it breaks the single component into less abstracted view of sub-system and modules and depicts their interaction with each other
  - Detailed design – deals with the implementation part e.g. logical structure, data structure, algorithms, etc.

Objective of Software Design

- **Correctness:** Software design should be correct as per requirement.
- **Completeness:** The design should have all components like data structures, modules, and external interfaces, etc.
- **Efficiency:** Resources should be used efficiently by the program.
- **Flexibility:** Able to modify on changing needs.
- **Consistency:** There should not be any inconsistency in the design.
- **Maintainability:** The design should be so simple so that it can be easily maintainable by other designers.

DESIGN PRINCIPLES

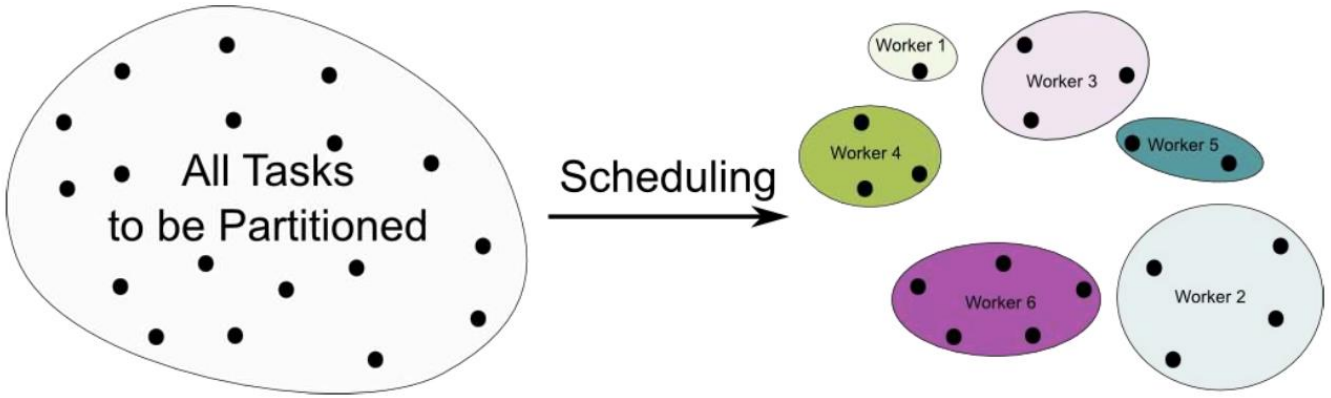
Software Design Principles

- Software design principles are concerned with providing means to handle the complexity of the design process effectively.

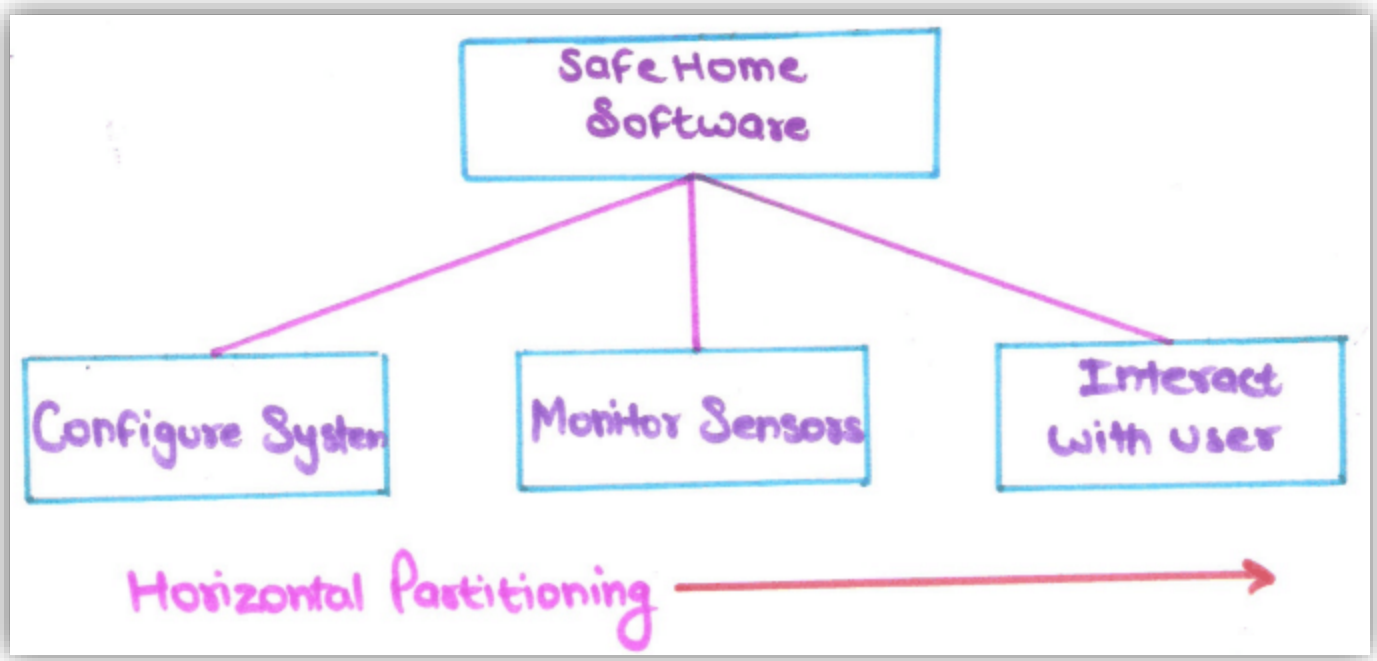
- Common software design principles
  - Problem partitioning
  - Abstraction
  - Modularity

Problem Partitioning

- **Divide into parts** that can be easily understood an established interfaces between the parts
- Divide the problem into smaller pieces so that each piece can be captured separately
- The goal is to divide the problem into manageable pieces

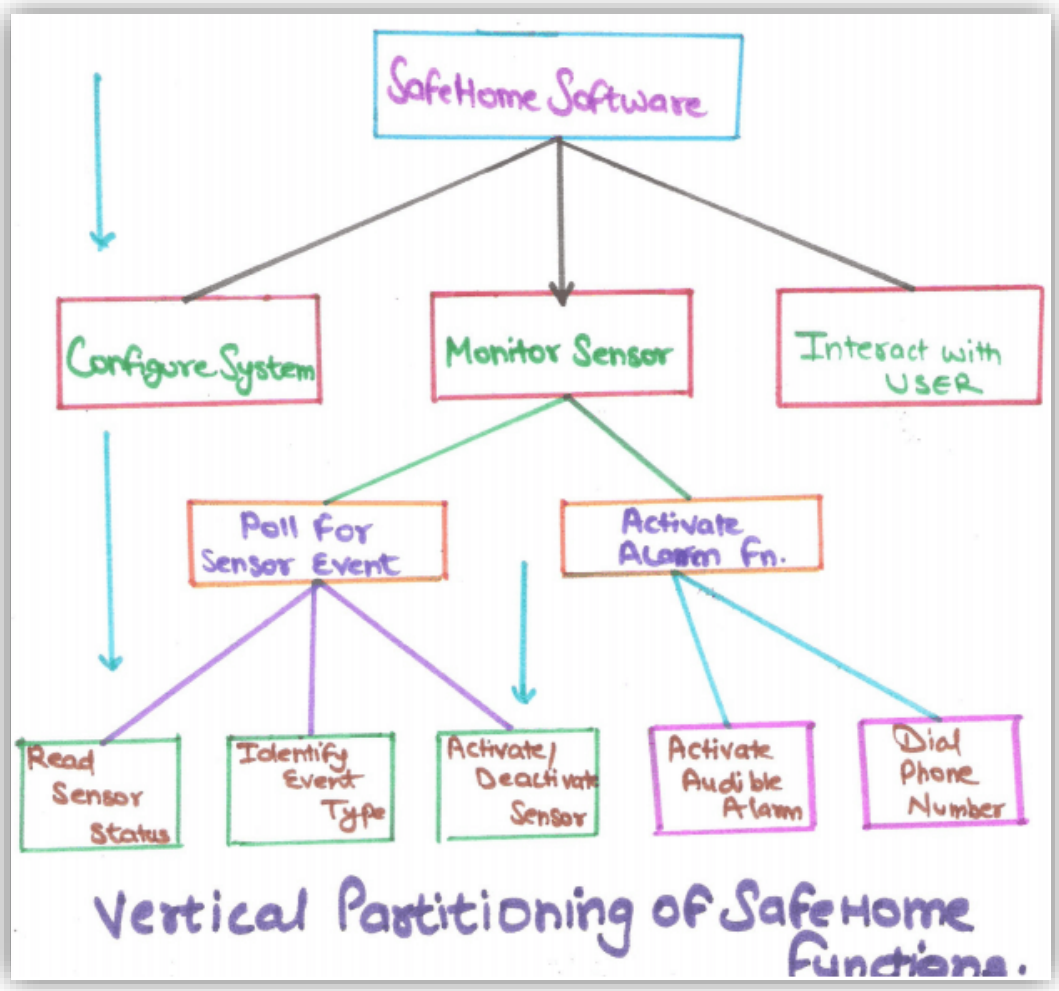


Problem Partitioning



Credit: http://www.tutorialsspace.com/Software-Engineering/SE-2nd-Unit/09-Analysis-Principles-Partitioning.aspx

Problem Partitioning



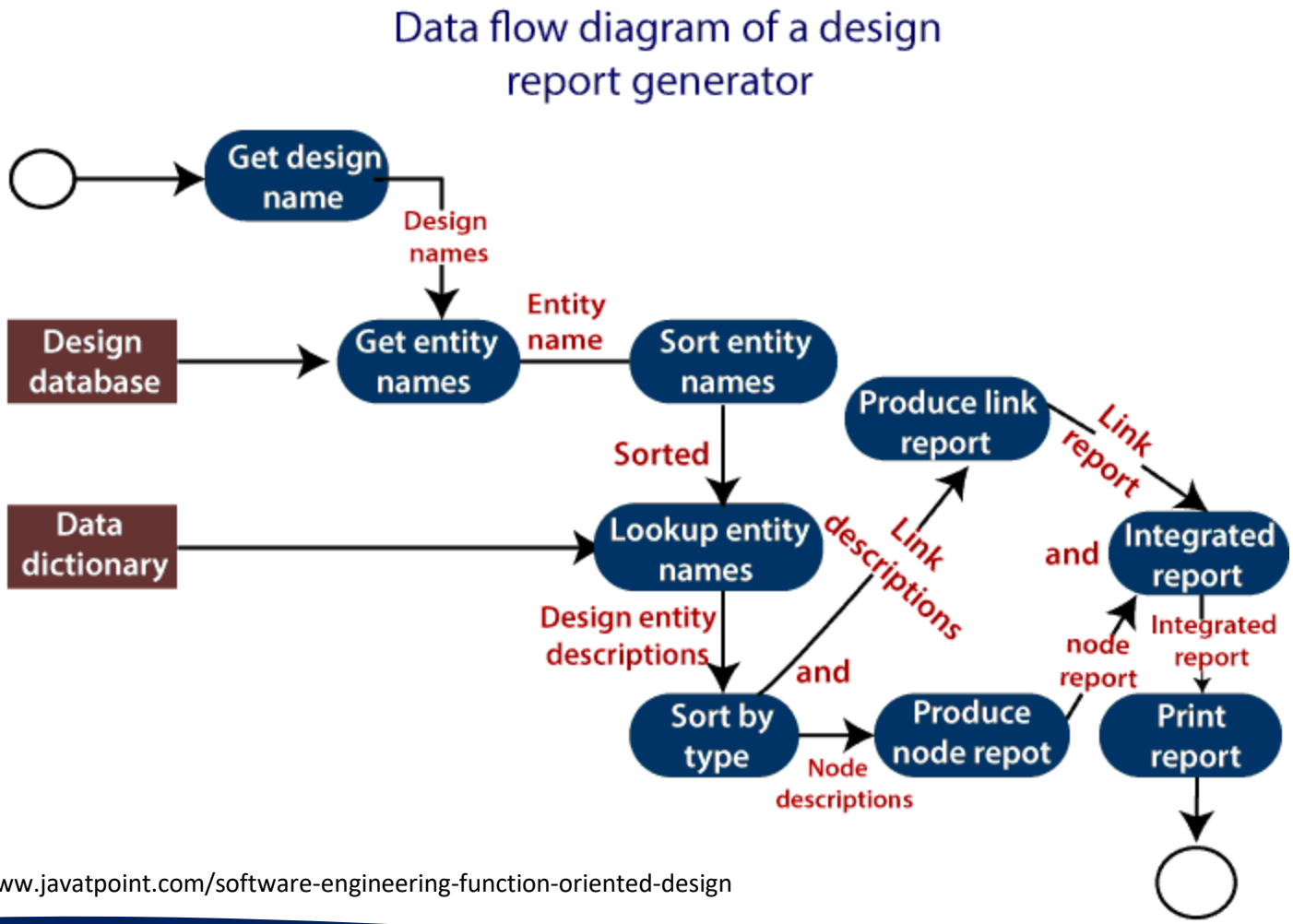
Credit: http://www.tutorialsspace.com/Software-Engineering/SE-2nd-Unit/09-Analysis-Principles-Partitioning.aspx

Abstraction

- Consider a component at an abstract level without bothering about the internal details of the implementation

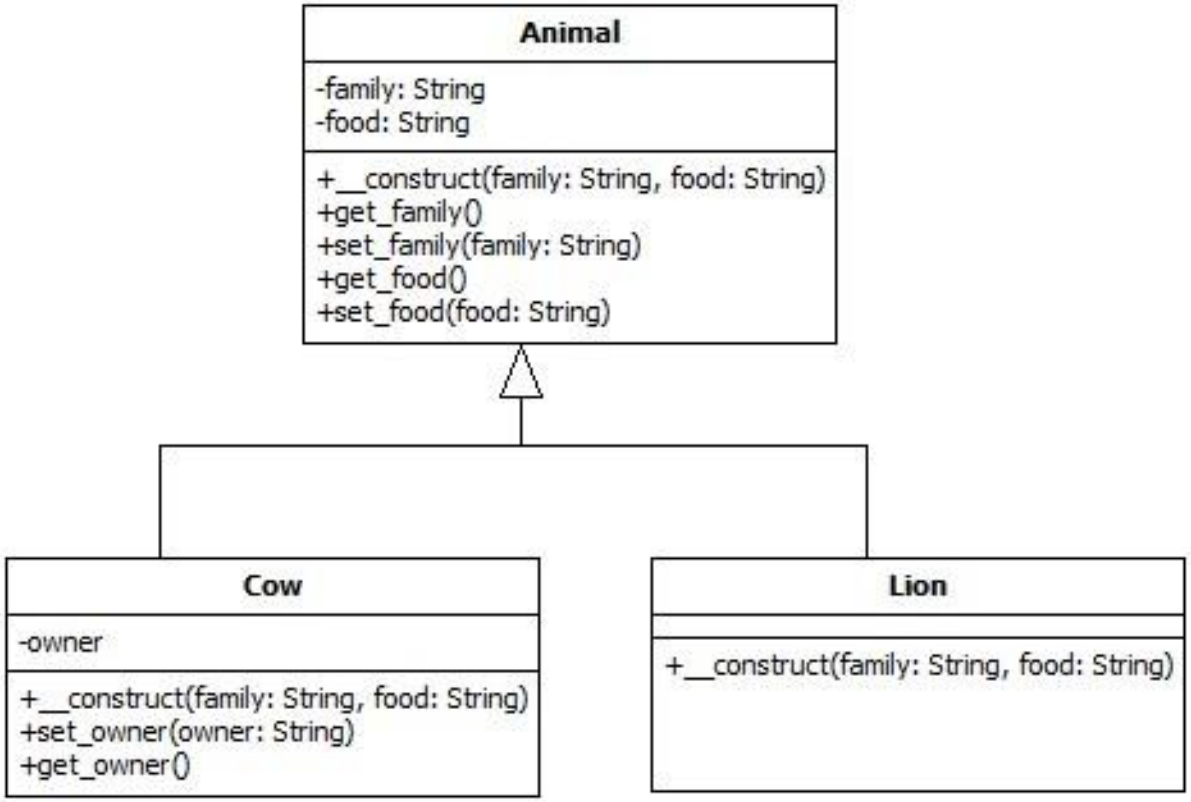
- Two common abstraction mechanisms
  - Functional abstraction – function oriented design approach (details of function not visible to user)
  - Data abstraction – object oriented design approach (details of data not visible to user)

Functional Abstraction



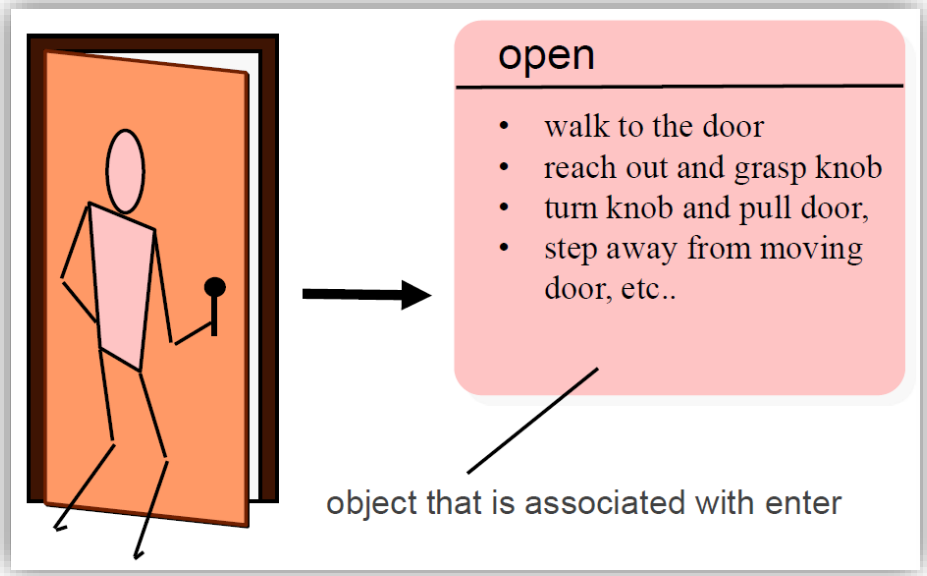
Credit: https://www.javatpoint.com/software-engineering-function-oriented-design

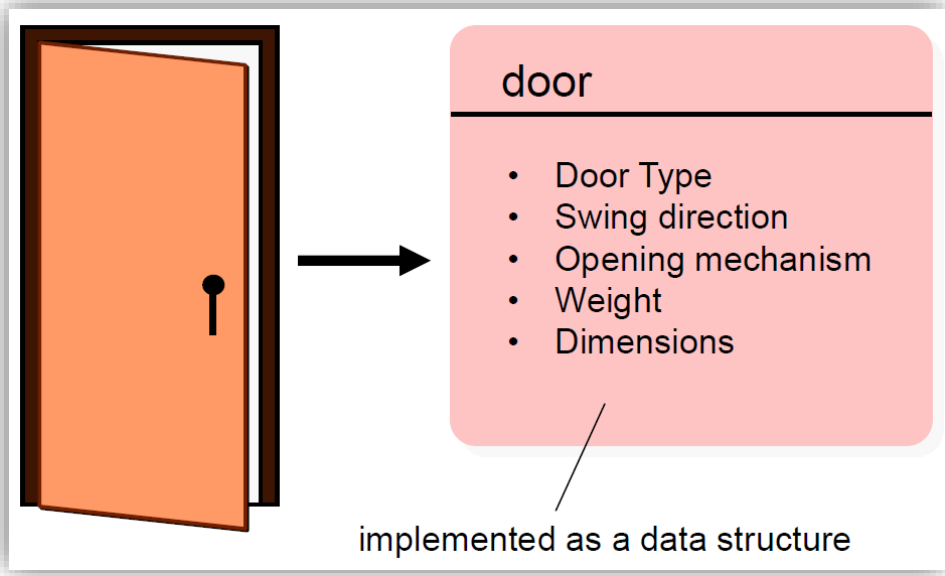
Data Abstraction



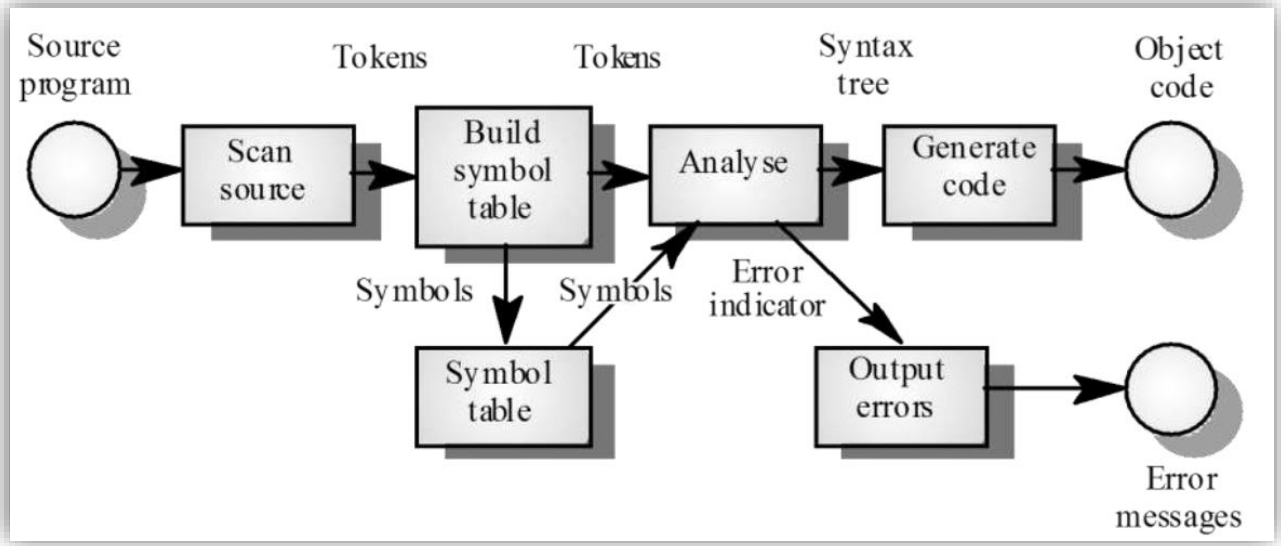
Credit: https://www.guru99.com/object-oriented-programming.html

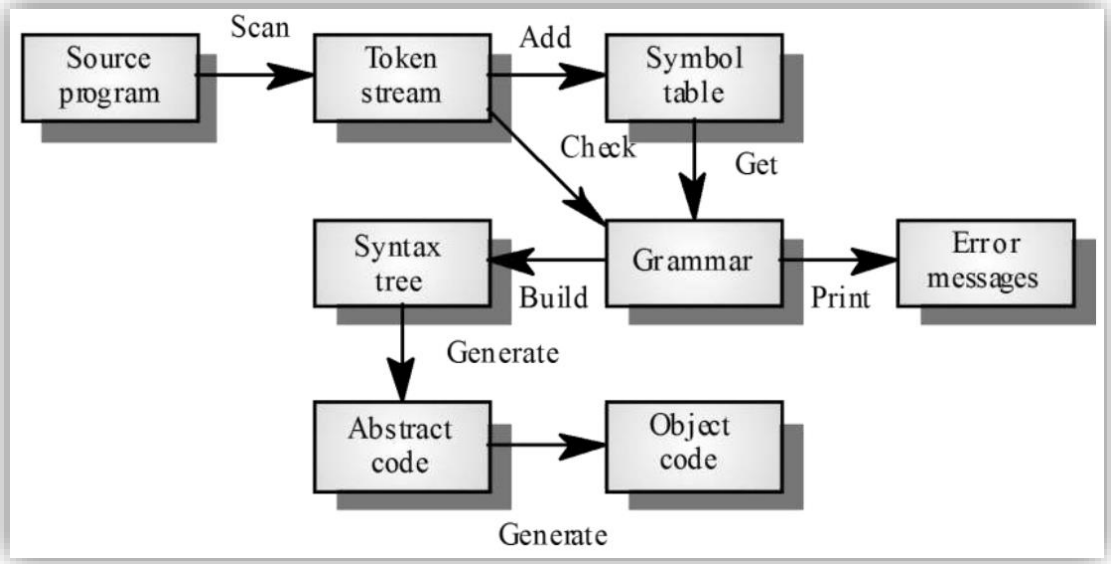
Functional Vs Data





Functional Vs Data
(View of A Compiler)

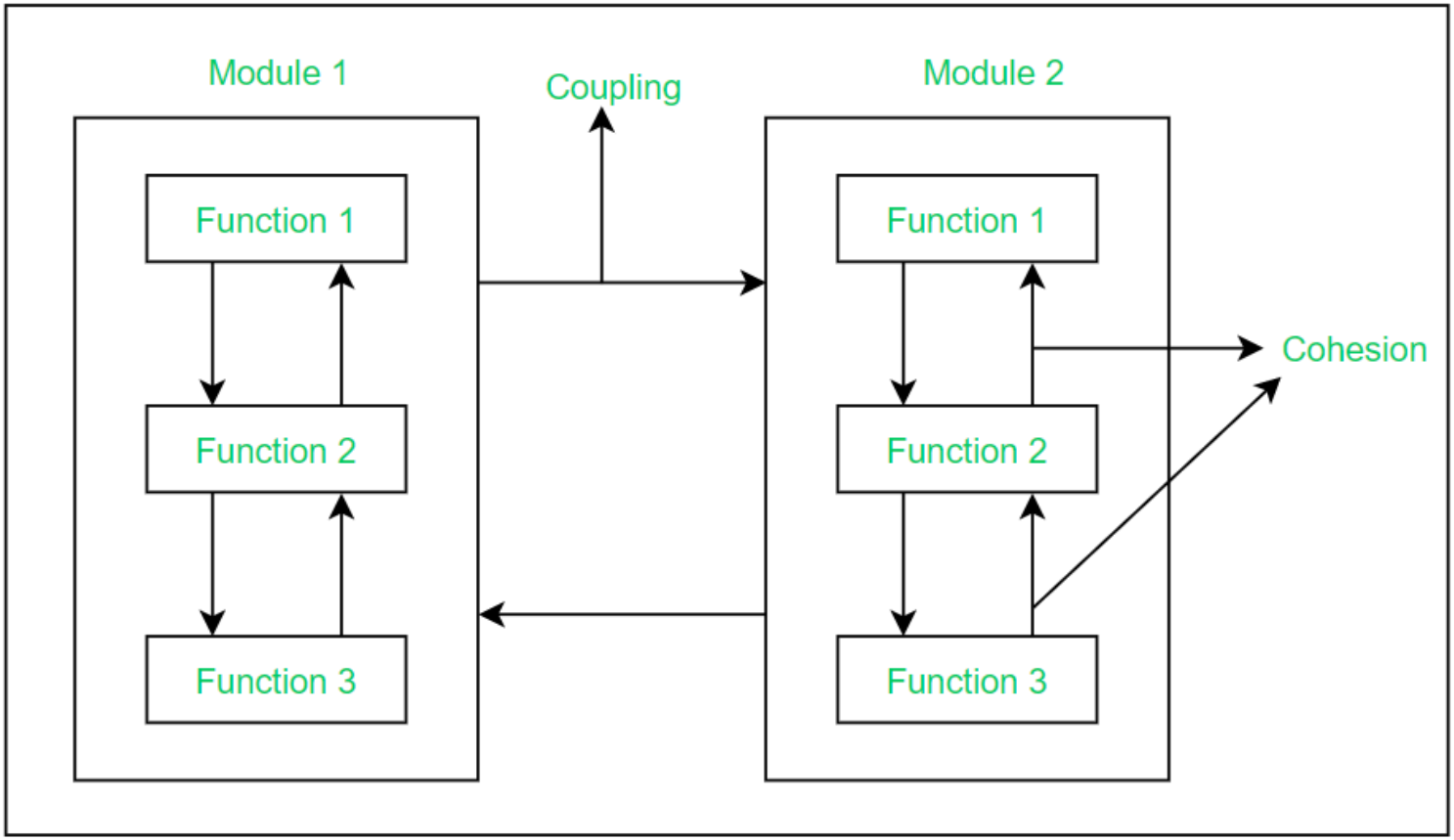




Modularity

- Modularity specifies to the division of software into separate modules
- Modules are named, addressed, and integrated later to obtain the complete functional software
- Each module is a well-defined system and has single specified objectives that can be used with other applications

Modularity

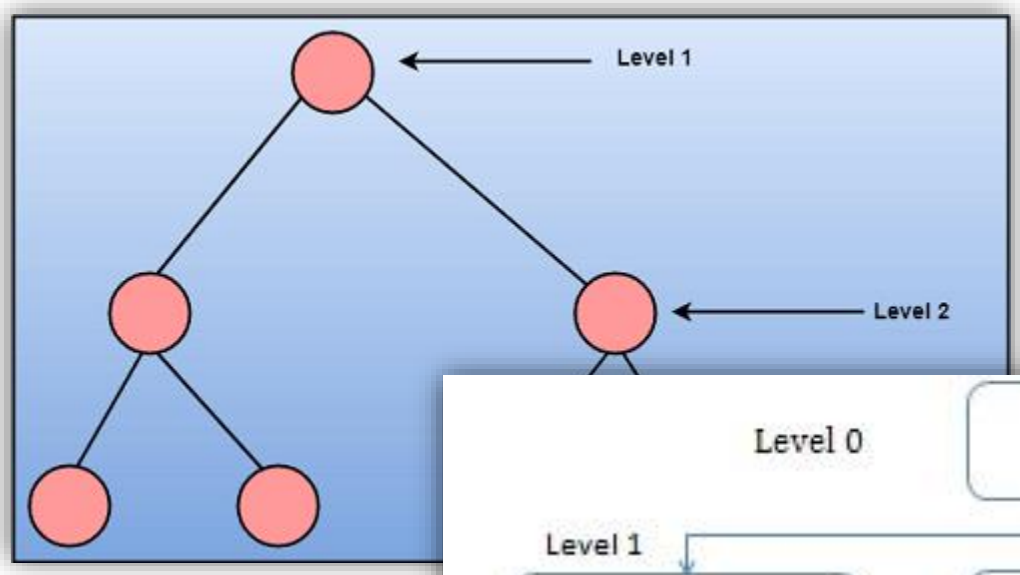


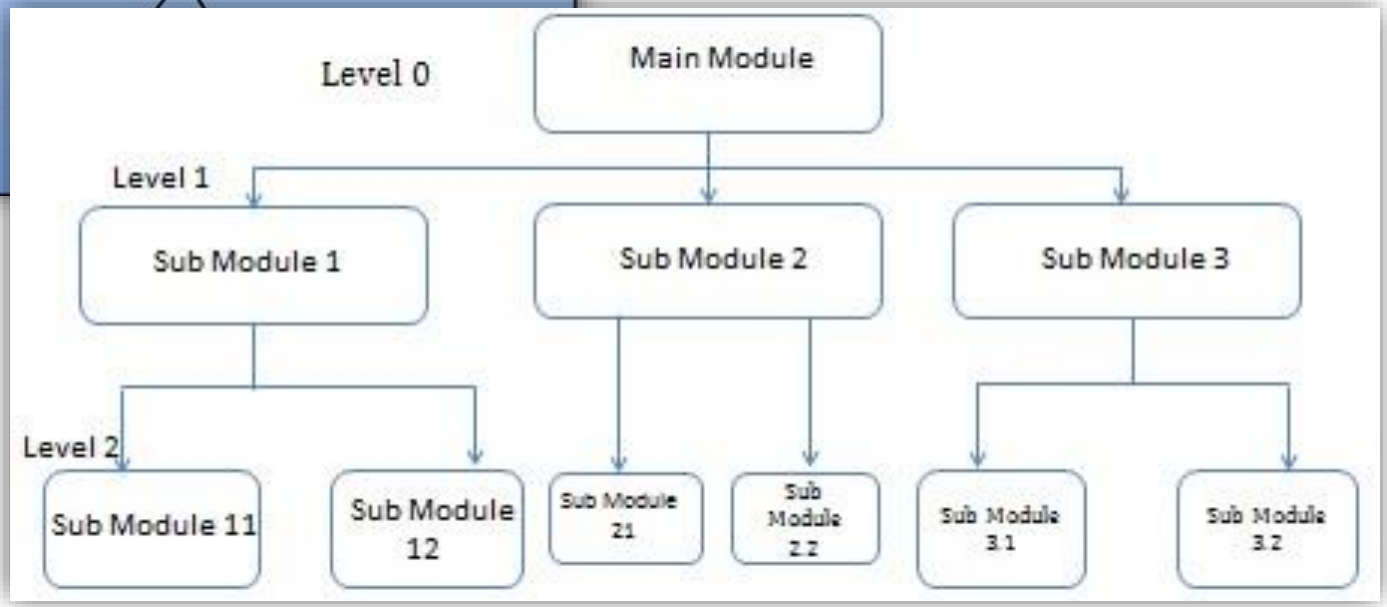
Credit: https://www.geeksforgeeks.org/effective-modular-design-in-software-engineering/

Strategy of Design

- Tow possible approaches
  - Top-down approach - starts with the identification of the main components and then decomposing them into their more detailed sub-components
  - Bottom-up approach - begins with the lower details and moves towards up the hierarchy, as shown in fig, suitable in case of an existing system

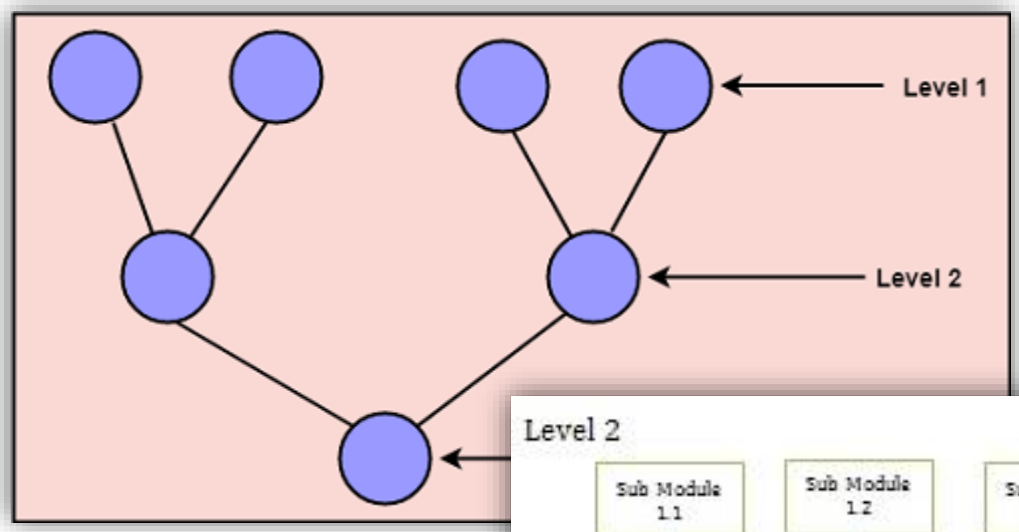
Top-down Approach

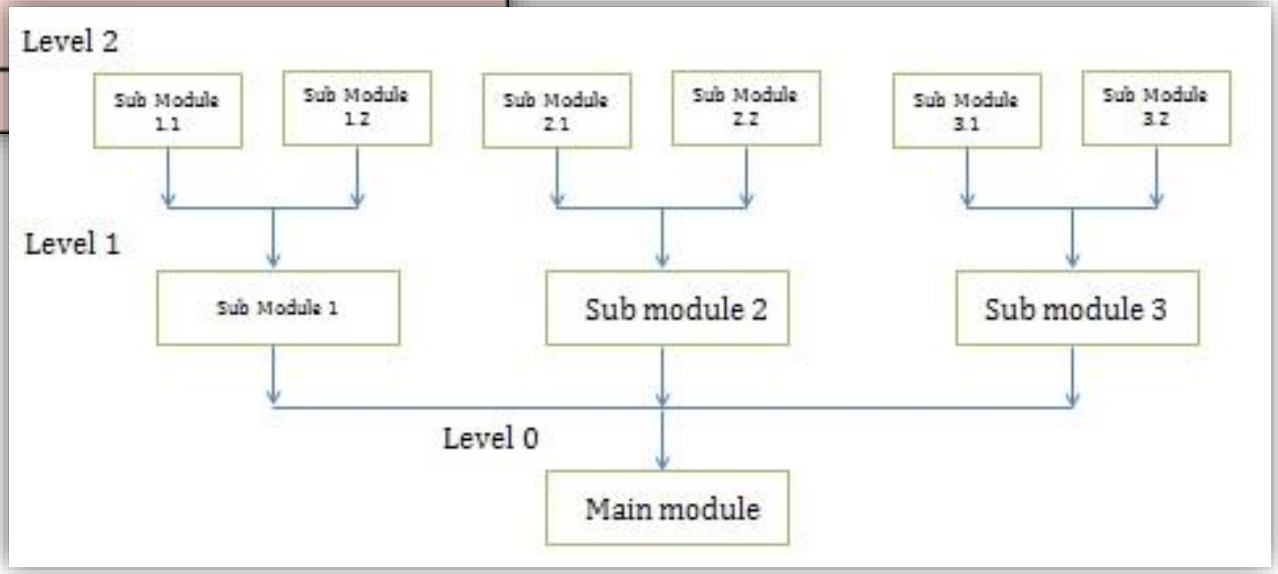




Credit: https://www.tutorialspoint.com/system_analysis_and_design/system_analysis_and_design_strategies.htm

Bottom-up Approach





Credit: https://www.tutorialspoint.com/system_analysis_and_design/system_analysis_and_design_strategies.htm

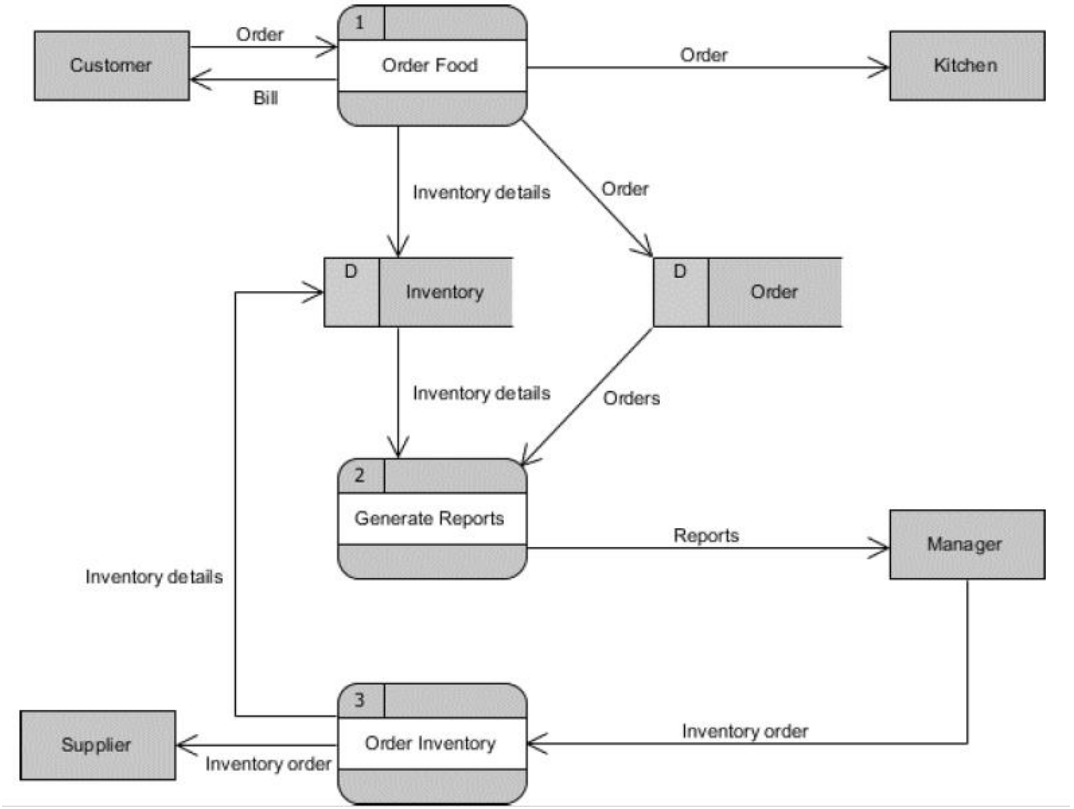
DESIGN NOTATION

Design Notation and Specification

- Modelling languages exist for facilitating software design tasks
  - Flowchart
  - UML
- Common diagrams
  - Dynamic
    - Data flow diagram (DFDs)
    - State transition diagrams (STDs) / state charts
    - Sequence diagram
  - Static
    - Entity relationship diagrams (ERDs)
    - Class diagrams
    - Structure charts
    - Object diagrams

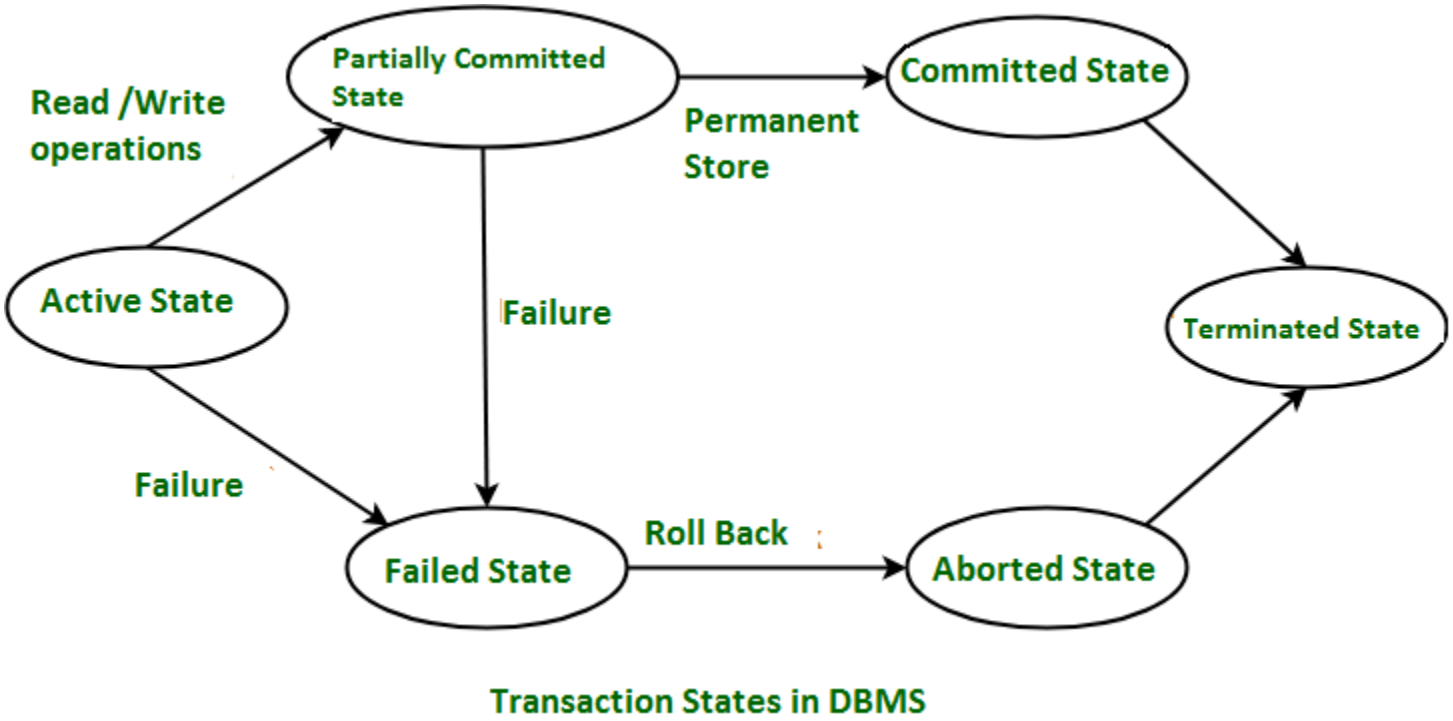
Data Flow Diagram

- Representing a flow of data through a process or a system



State Transition / Statechart Diagram

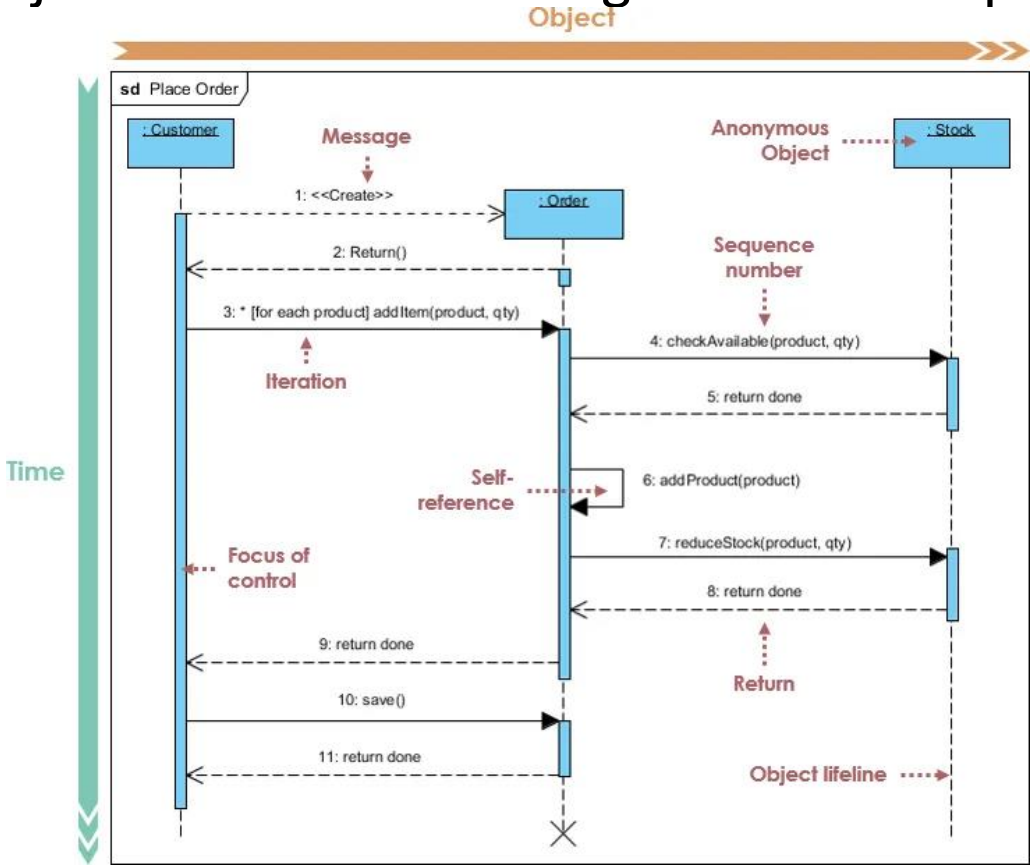
- Shows various states of a single object which a transaction goes during its lifetime



Credit: https://www.geeksforgeeks.org/transaction-states-in-dbms/

Sequence Diagram

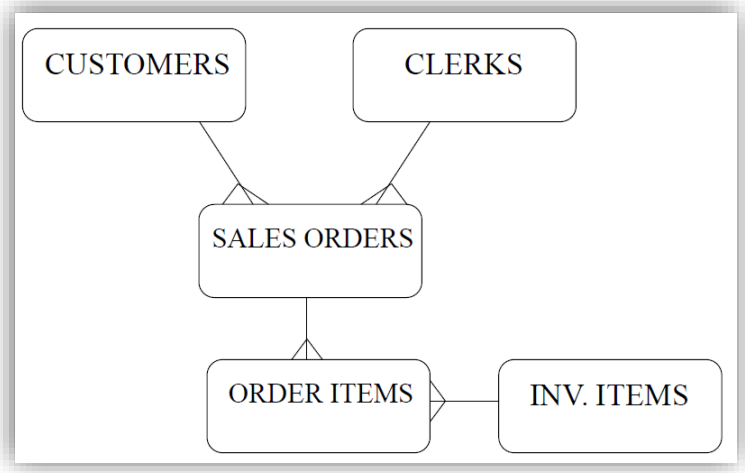
- Showing object interactions arranged in time sequence

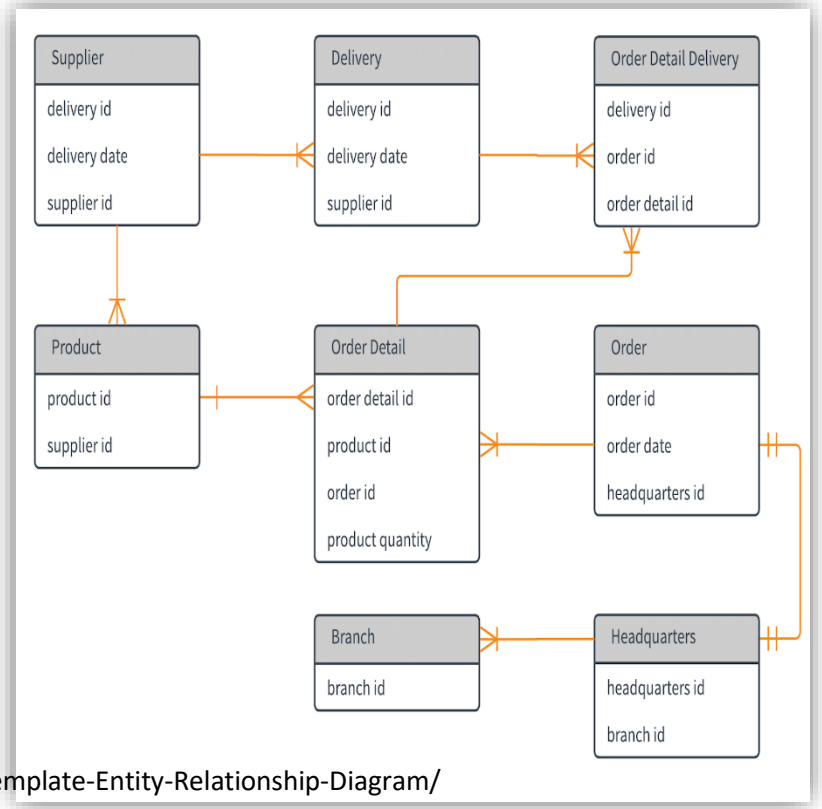


Credit: https://www.archimetric.com/what-is-sequence-diagram/

Entity Relationship Diagram

- Represents conceptual level of a database system which describes things and their relationship in high level

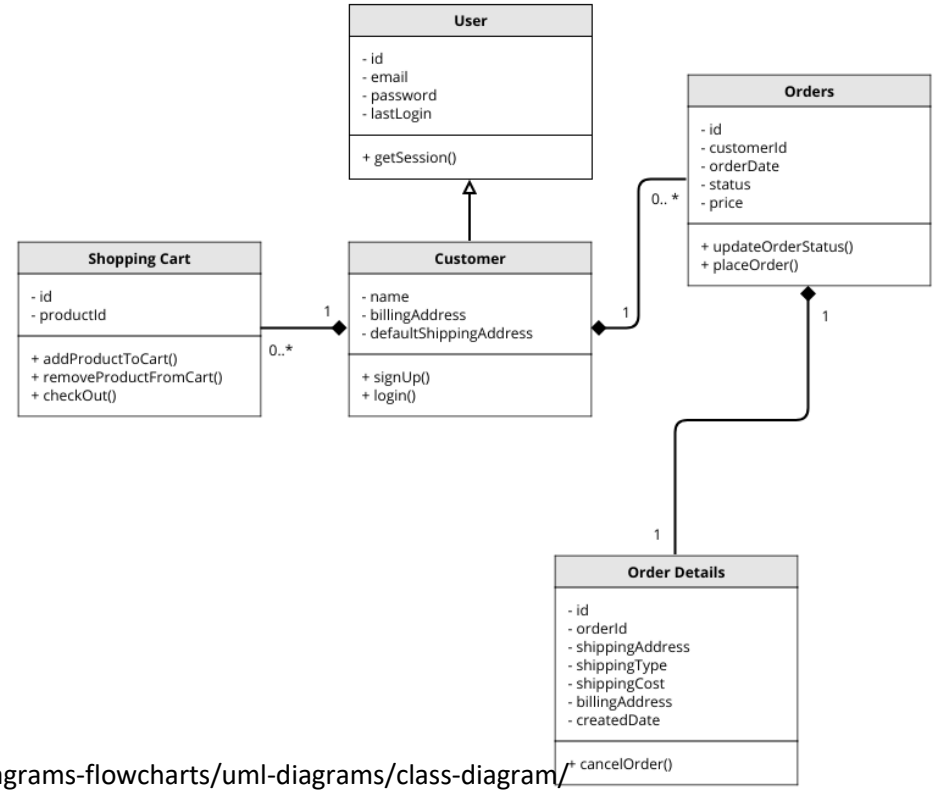




Credit: https://lucidchart.zendesk.com/hc/en-us/articles/360000478183-Template-Entity-Relationship-Diagram/

Class Diagram

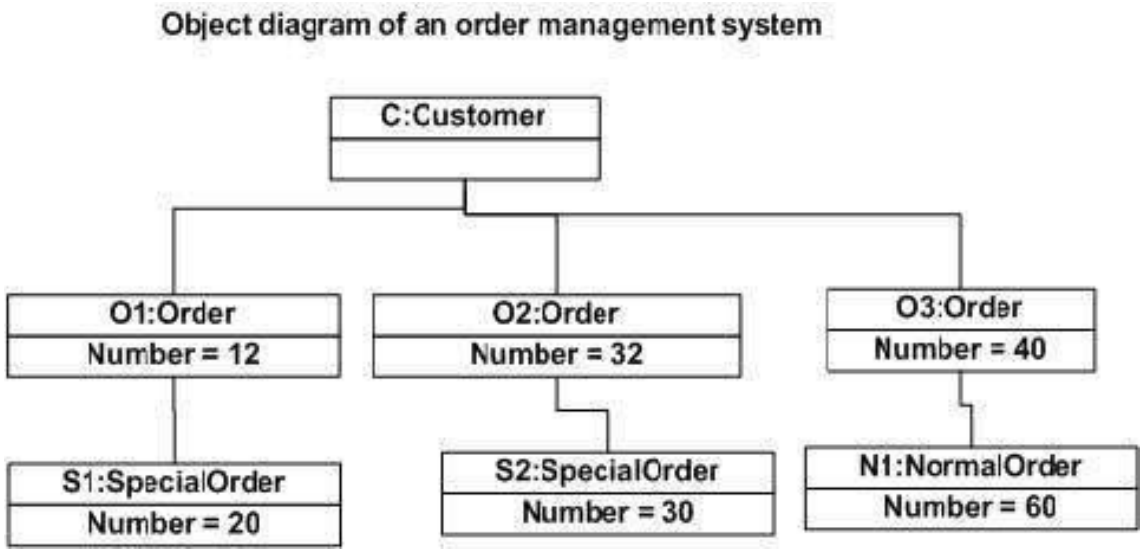
- Describing the structure of a system showing classes, attributes, operations (or methods), as well as relationships among classes



Credit: https://moqups.com/templates/diagrams-flowcharts/uml-diagrams/class-diagram/

Object Diagram

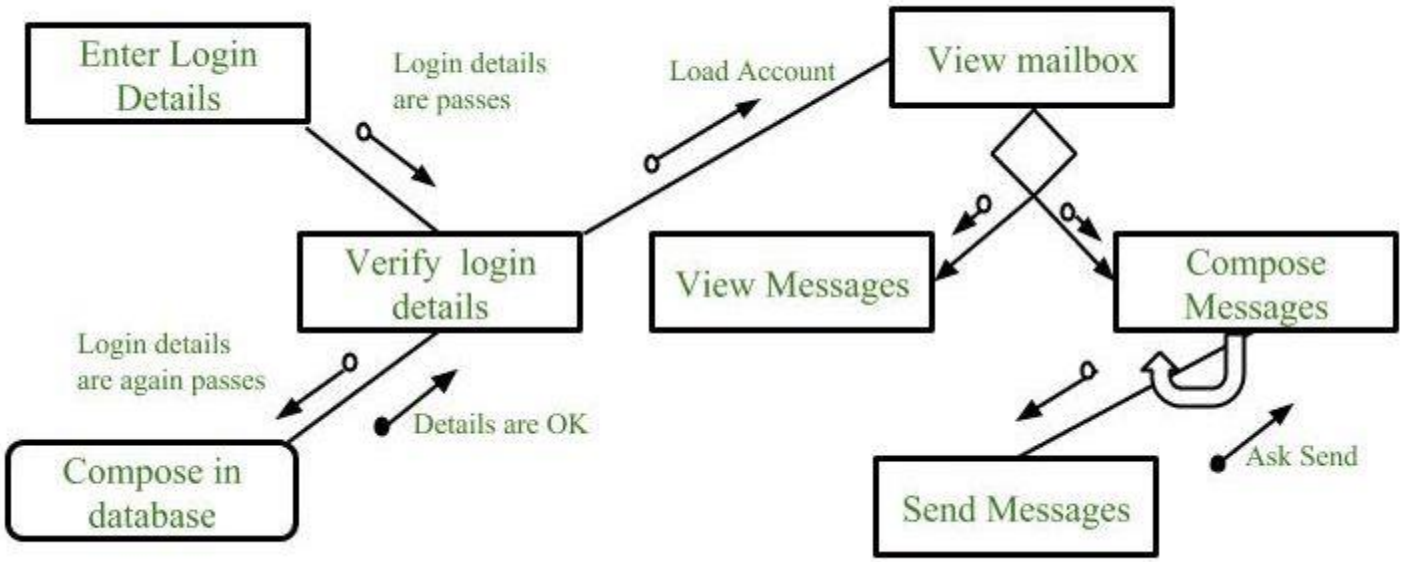
- Derived from class diagrams so object diagrams are dependent upon class diagrams
- Represent the static view of a system but this static view is a snapshot of the system at a particular moment



Credit: https://www.tutorialspoint.com/uml/uml_object_diagram.htm

Structure Chart

- Breaks down the entire system into lowest functional modules, describe functions and sub-functions of each module of a system to a greater detail



Credit: https://www.geeksforgeeks.org/software-engineering-structure-charts/

USER INTERFACE DESIGN

User Interface

- System users often judge a system by its interface rather than its functionality
- A poorly designed interface can cause a user to make catastrophic errors
- Poor user interface design is the reason why so many software systems are never used
- Good user interface
  - They are easy to learn and use. Users without experience can learn to use the system quickly
  - User may switch quickly from one task to another and can interact with several different applications

UI Design Principles

- User familiarity
  - The interface should be based on user-oriented terms and concepts rather than computer concepts. For example, an office system should use concepts such as letters, documents, folders etc. rather than directories, file identifiers, etc.
- Consistency
  - The system should display an appropriate level of consistency. Commands and menus should have the same format, command punctuation should be similar, etc.
- Minimal surprise
  - If a command operates in a known way, the user should be able to predict the operation of comparable commands

UI Design Principles

- Recoverability
  - The system should provide some resilience to user errors and allow the user to recover from errors. This might include an undo facility, confirmation of destructive actions, 'soft' deletes, etc.
- User guidance
  - Some user guidance such as help systems, on-line manuals, etc. should be supplied
- User diversity
  - Interaction facilities for different types of user should be supported. For example, some users have seeing difficulties and so larger text should be available

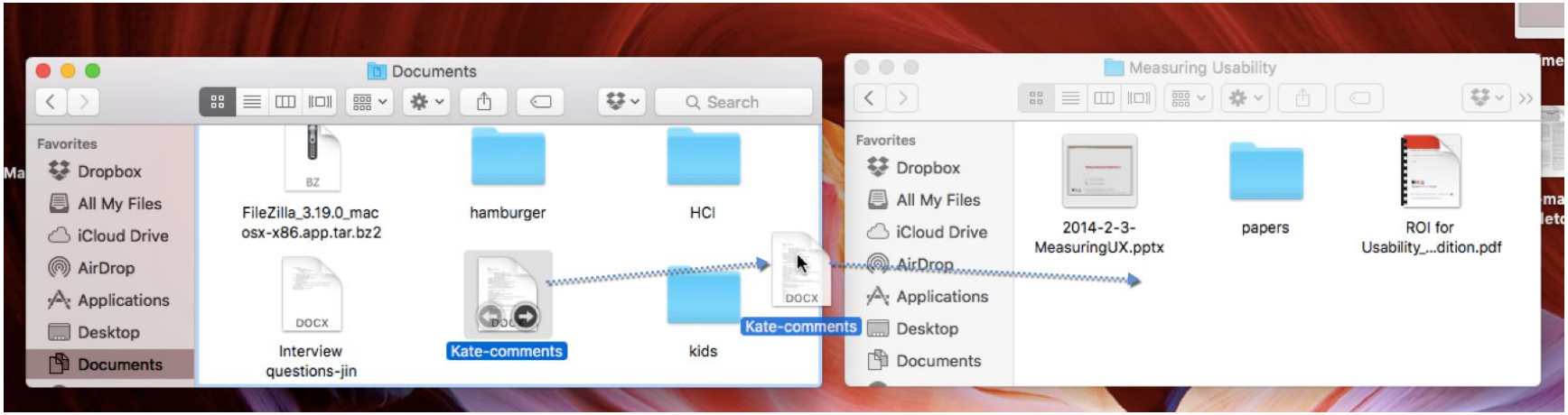
Interaction Style

- Direct manipulation
- Manu selection
- Form fill-in
- Command language

Direct Manipulation

- An interaction style in which users act on displayed objects of interest using physical, incremental, reversible actions whose effects are immediately visible on the screen.
- Advantage
  - Users feel in control and less likely to be intimidated by it
  - Short learning time
  - Users get immediate feedback on their actions so mistakes can be quickly detected and corrected

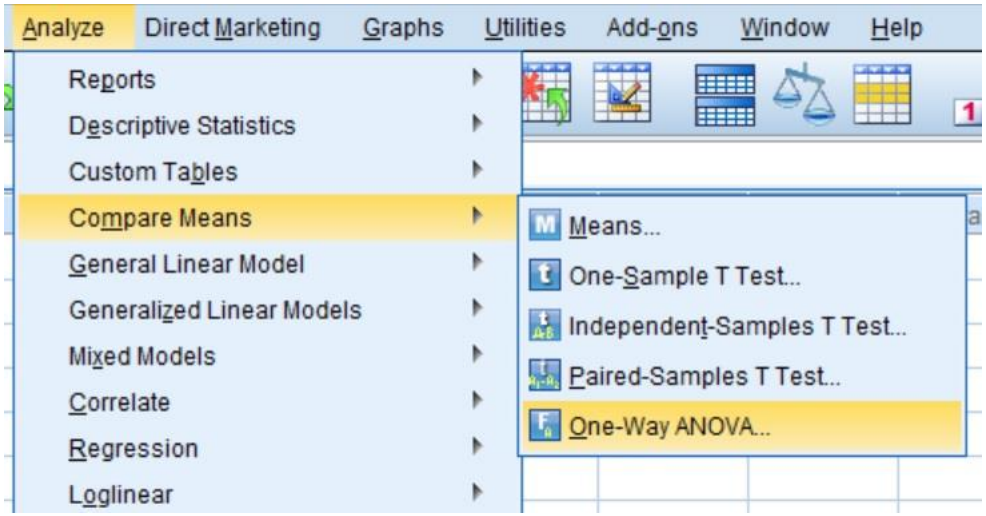
Direct Manipulation



Credit: https://www.nngroup.com/articles/direct-manipulation/

Menu Selection

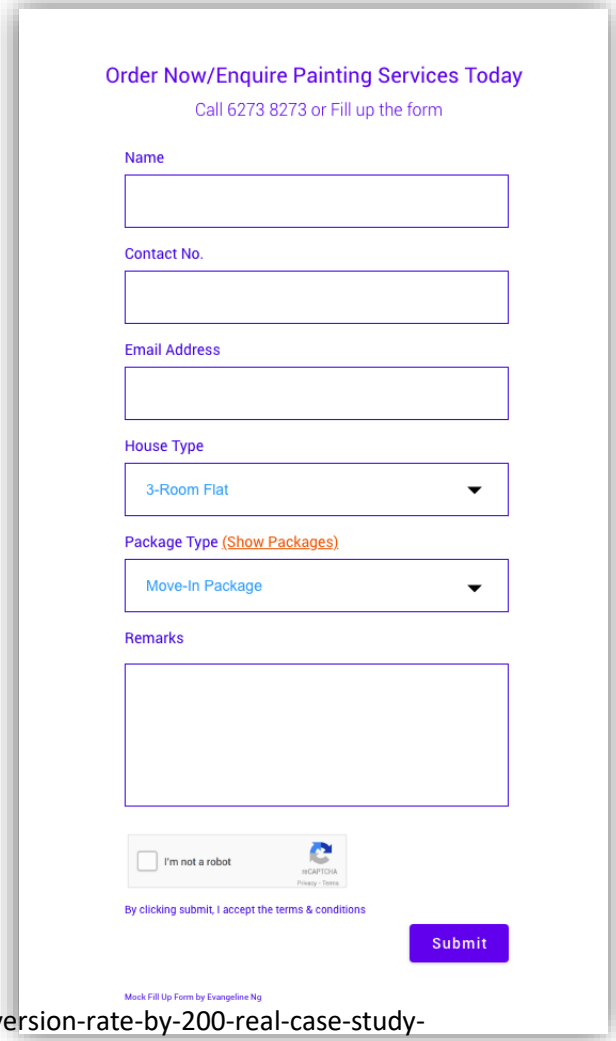
- The selection may be made by pointing and clicking with a mouse, using cursor keys or by typing the name of the selection – avoids user error, little typing required
- Users make a selection from a list of possibilities presented to them by the system



Credit: https://toptipbio.com/perform-one-way-anova-spss/one-way-anova-menu-selection/

Form Fill-in

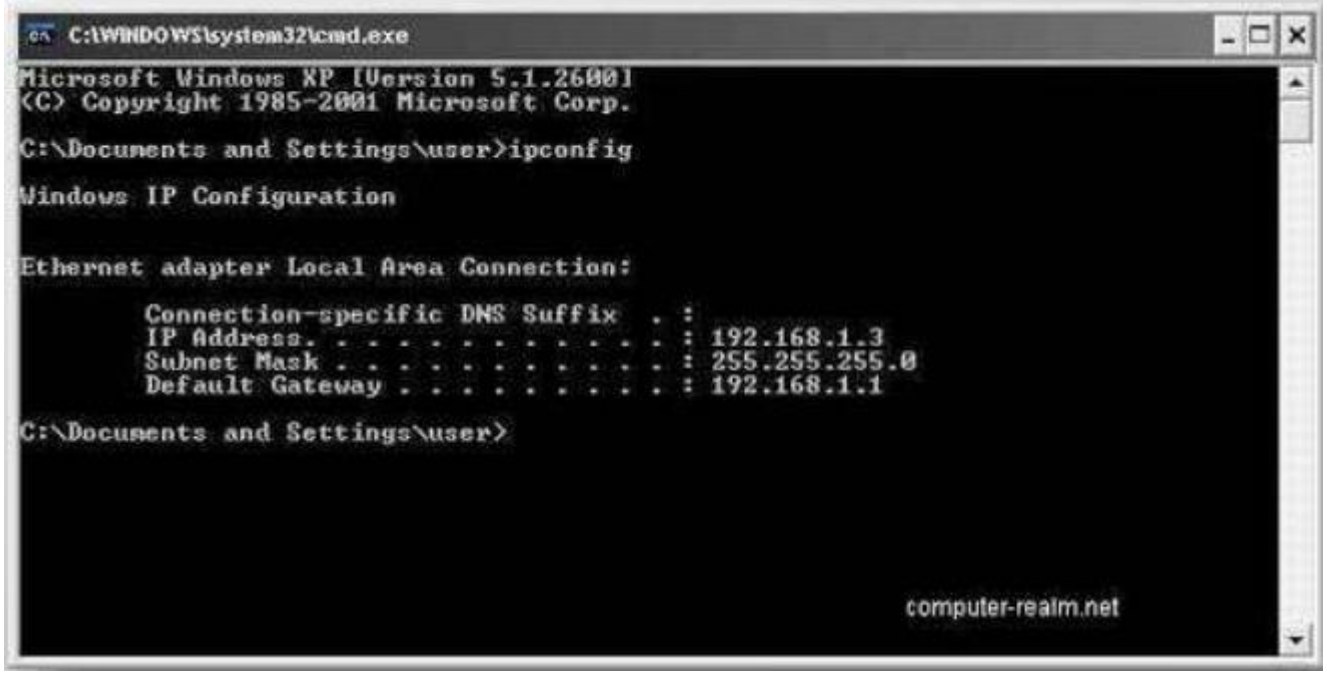
- In a form-based interface, the user interacts with the application by selecting one of a number of possible values, and by entering text into the fields that accept it
- Simple data entry and easy to learn



Credit: https://blog.prototypr.io/ux-how-to-optimise-a-fill-in-form-that-will-increase-conversion-rate-by-200-real-case-study-cda7b8a48295

Command Language

- User types commands to give instructions to the system
- Powerful and flexible but hard to learn

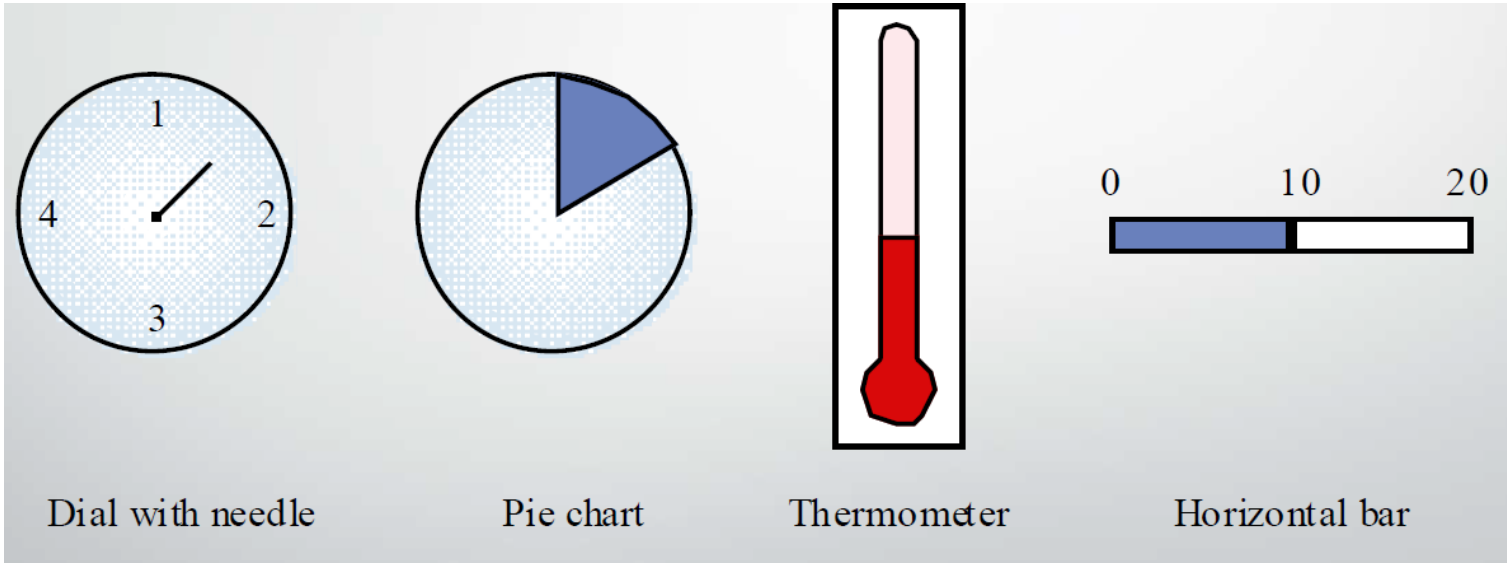


Information Presentation

- Information presentation is concerned with presenting system information to system users
- The information may be presented directly (e.g. text in a word processor) or may be transformed in some way for presentation (e.g. in some graphical form)
- Dealing with many concerns
  - Data visualization
  - Color displays
  - Error displays
  - Etc.

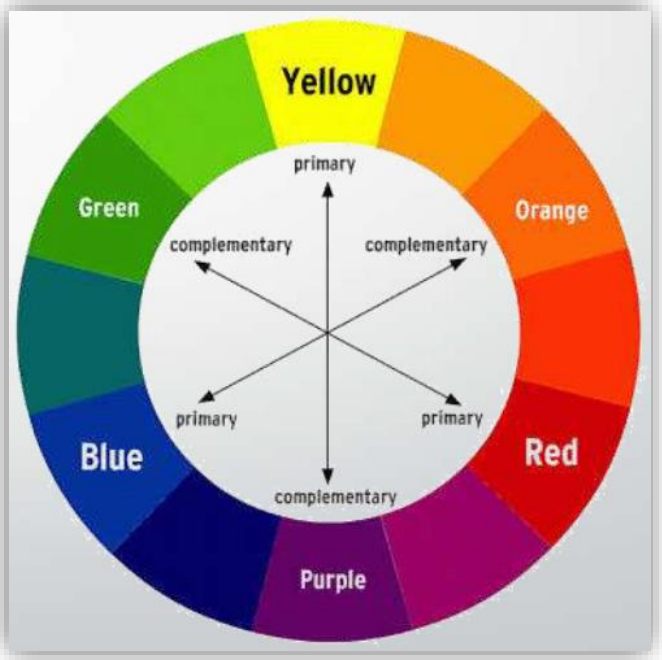
Data Visualization

- Concerned with techniques for displaying large amounts of information
- Visualisation can reveal relationships between entities and trends in the data



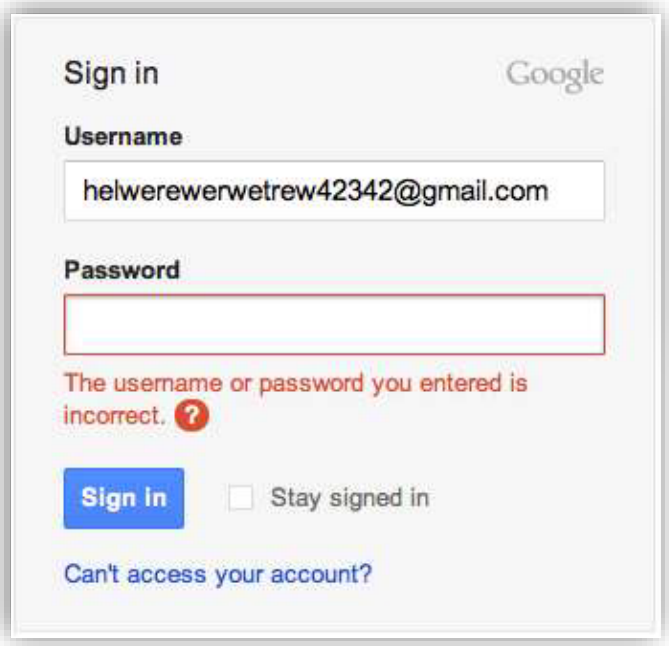
Color Display

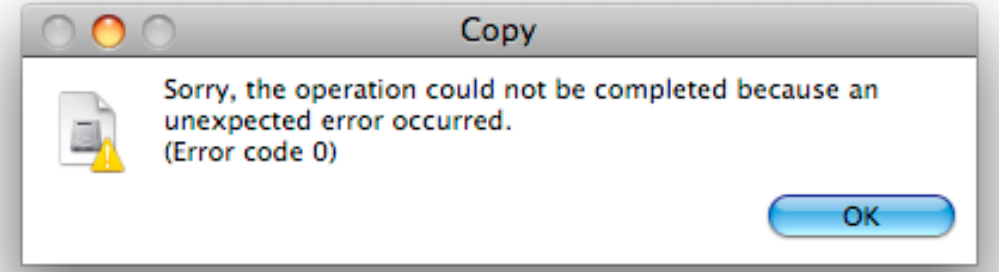
- Don't use too many colors
- Use color coding to support user tasks
- Design for monochrome then add color
- Use color coding consistently
- Use color change to show status change



Error Display

- Error message design is critically important
- Messages should be polite, concise, consistent and constructive





Credit: https://www.hung-truong.com/blog/2009/02/11/who-says-windows-should-get-all-the-bad-error-messages/

User Interface Evaluation

- Learnability – how long does it take a new user to become productive with the system?
- Speed of operation – how well does the system response match the user's work practice?
- Robustness – how tolerant is the system of user error?
- Recoverability – how good is the system at recovering from user errors?
- Adaptability – how closely is the system tied to a single model of work?

Exercise 3.1

- In your home or workplace find a simple machine, something with two to four controls and with no (!) internal computer (a simple toaster or an electric drill are possible candidates). Describe:
  - the users the machine seems to be designed for;
  - the tasks and subtasks the machine was evidently designed to support;
- Explain how the machine is designed for support users and tasks

Credit: https://hcibib.org/tcuid/exercises.html

Exercise 3.2

- Define system architecture and functions/components in your project.

Credit: https://hcibib.org/tcuid/exercises.html